CS04391: Parallel and Concurrent Computing

Final Project

Name: Ch. Muhammad Shaheer Yasir

Banner ID: 916518314

1. Problem Specification

In many networks, whether social, biological, or technological, researchers look for tightly knit groups of nodes that interact with one another more densely than with the rest of the network. Traditionally, these groups are modeled as cliques, where every member is directly connected to every other member. For example, in Figure 1, the highlighted orange portion in (b) and (c) are cliques as all the nodes are connected to every other node. However (b) is not a maximal clique as it is part of (c) which is a clique. Figure 1(a) is not a clique as the top and bottom nodes are missing 1 connection. In practice, however, perfect cliques seldom appear in real-world data: missing or noisy connections often break the complete "everyone-knows-everyone" structure.

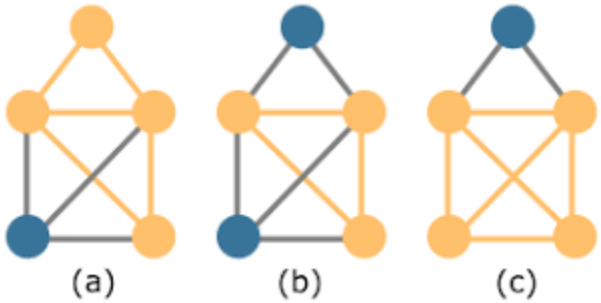

Figure 1: Concept of a Clique

To allow for a bit of flexibility, we turn to the concept of a k-plex. In a k-plex, each member is permitted to miss connections with up to k other members. A 1-plex is just a clique, while larger k values relax the requirement ever further. For example, in Figure 2, nodes {1,2,3,4,5} and {1,2,3,4,6} are 2-plexes as nodes are missing at max 2 connections: 1 itself and one

other node. A maximal k-plex is one that can't be grown any larger without violating that at most k missing links per node rule. For example, in Figure (2), nodes {1,2,3,4} are making a k-plex but they cannot form a maximal k-plex as they're part of other k-plexes {1,2,3,4,5} and {1,2,3,4,6}. Because they tolerate a few weak or missing ties, k-plexes tend to reveal more realistic communities in messy real-world networks, and they've been used successfully in everything from detecting social circles to uncovering functional modules in protein-interaction networks.

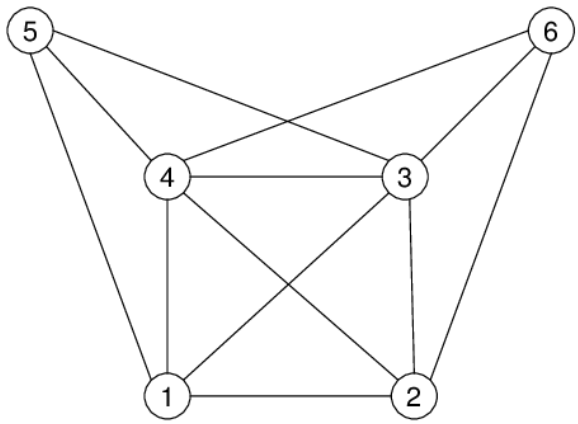

Figure 2: Concept of a k-plex

The computational challenge lies in enumerating all maximal k-plexes above a given size threshold q: there can be an astronomical number of them, and the problem is NP-hard in general. Building on the classic Bron–Kerbosch backtracking framework for clique enumeration, a recent branch-and-bound algorithm (2025) [1] cleverly prunes the search space to find large k-plexes more efficiently. In our project, we took the next step toward speed: we parallelized the decomposition phase of this branch-and-bound method on NVIDIA GPUs using CUDA. By offloading the work of breaking the graph into smaller subproblems across thousands of GPU threads, we've dramatically accelerated the search enabling timely analysis of even very large networks.

2. Program Design

2.1. Program Overview

The original algorithm follows a decomposition process before starting the real recursive process. The decomposition process involves dividing the original graph into multiple subgraphs and assigning each subgraph to one core of the CPU. Each of these subgraphs is

the direct and 2-hop neighborhood of each vertex in the graph. Figure (3) represents a subgraph induced by a vertex v_i.

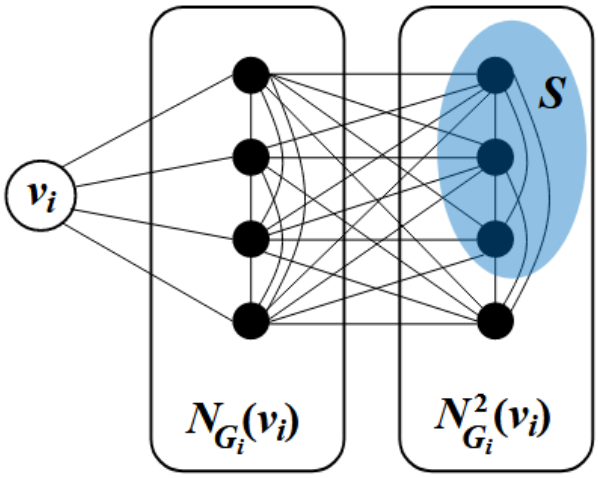

Figure 3: Subgraph induced by seed vertex v_i

There are many strategies followed by the original algorithm to decrease the search space quickly. Some of them that we implemented in our CUDA version are as follows:

1. **Graph Pruning Strategies:** There are many pruning strategies the original algorithm implemented to reduce the search space. Foremost is reducing the original graph to its q-k core. It will prune all the nodes having degree less than q. The second strategy is related to the total number of common neighbors and common 2-hop neighbors. It is based on a theorem they proved in their paper that is if the number of common neighbors between the seed vertex and any node in its subgraph is less than q-2k, then that node should be pruned. Similarly, if the number of two-hop neighbors is less than q-2k+2, it will be pruned.
2. **Degeneracy Ordering:** The algorithm is sorting all the nodes by their degeneracy so that the nodes with the least degree will be processed first. To sort the graph by its degeneracy order, the algorithm is using List Linear Heap with key-value pairs. The key represents the degree of the node, and the value is a Linked List having all the nodes stored with that degree. In this way, nodes with the least degree appear first as they'll have a lower key.
3. **Pivot Based Exploration:** The pivot is chosen as the node with the least degree (that comes first in the degeneracy order). Also, all those nodes appearing before the seed node isn't considered for subgraph generation but only used when checking

maximality. Also, the loop is stopped early when the number of nodes after the current seed vertex is less than q.

The parallel version of the original algorithm is using OpenMP to divide all the seed vertices (labeled as tasks) among different cores of a CPU. Figure 4 shows the division of tasks into different cores based on seed vertices.

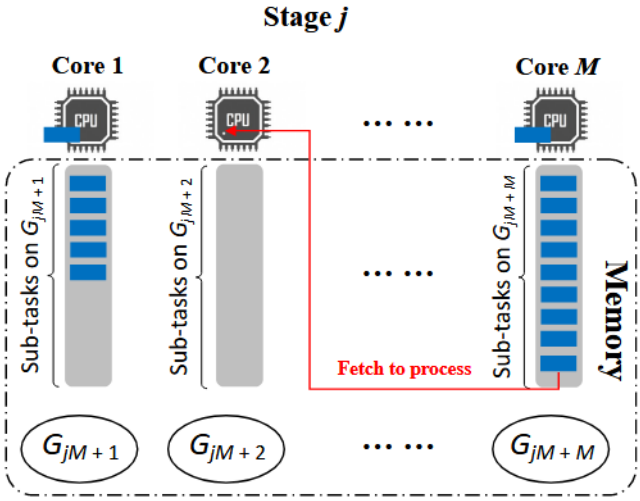

Figure 4: Illustration of Parallel Processing of k-plex Enumeration

2.2. Our Contribution

We kept most of the logic of the original algorithm intact while implementing it on CUDA to utilize the huge parallelization GPUs offer to find maximal k-plexes in graphs in less time. However, keeping the memory constraints in GPU in consideration, we changed the graph storage method from class based to Compressed Sparse Row (CSR) Format. In this format, we created arrays of offsets, neighbors, and degrees. Here offsets indicate the start and end index of neighbors of each node in neighbors' array, while the neighbors array store all the neighbors of each node. Also, we will assign each seed vertex to one warp of the GPU so that we can utilize the parallelism of 32 threads while computing the logic of subgraph generation of one seed vertex.

We started by reducing the original graph to its q-k core using peelGraph() function. It was called from the host side as the process was using queues to remove all the nodes that aren't satisfying the q-k core condition. However, we had to modify the original peelGraph function to accommodate the CSR format of our graph.

Then, we sorted the original graph according to its degeneracy ordering using List Linear Heap. This was also computed on the host side using a single core due to higher dependencies.

Next, we started declaring memory, allocating, and setting memory on the device side for our kernel. First, we defined three structures in common.h that will be used to transfer data from CPU to GPU. Figure 5 shows the structures we created using struct keyword.

```
typedef struct P_pointers{
    int k;
    int lb;
    int bd; //q-k
} P_pointers;

typedef struct G_pointers{
    unsigned int n, m;
    unsigned int *offsets;
    unsigned int *neighbors;
    unsigned int *degree;
} G_pointers;

typedef struct D_pointers{
    int *dpos;
    int *dseq;
} D_pointers;
```

Figure 5: Structures created to store device data

The P_pointers store the basic information such as k, q(lb), and bd(q-k), while G_pointers stores the original graph in CSR format. Moreover, D_pointers store the degeneracy ordering where dseq represents the sorted nodes, while dpos represents the position of each node in the sorted sequence.

Next, we allocated and transferred all of these structures to GPU using the function copy_graph_to_gpu. This function is using cudaMalloc to allocate a certain memory on GPU and then use cudaMemcpy to transfer data from host to device side. Figure 6 illustrates the copy_graph_to_gpu function.

```
// Copying original graph to GPU
chkerr(cudaMalloc(&(g.offsets), (peelG.n + 1) * sizeof(unsigned int)));
chkerr(cudaMalloc(&(g.neighbors), (peelG.m) * sizeof(unsigned int)));
chkerr(cudaMalloc(&(g.degree), (peelG.n) * sizeof(unsigned int)));

chkerr(cudaMemcpy(g.offsets, peelG.offsets, (peelG.n + 1) * sizeof(unsigned int), cudaMemcpyHostToDevice));
chkerr(cudaMemcpy(g.neighbors, peelG.neighbors, (peelG.m) * sizeof(unsigned int), cudaMemcpyHostToDevice));
chkerr(cudaMemcpy(g.degree, peelG.degree, (peelG.n) * sizeof(unsigned int), cudaMemcpyHostToDevice));

g.n = peelG.n;
g.m = peelG.m;

chkerr(cudaMalloc(&(d.dpos), (peelG.n) * sizeof(int)));
chkerr(cudaMalloc(&(d.dseq), (peelG.n) * sizeof(int)));

chkerr(cudaMemcpy(d.dpos, dpos, (peelG.n) * sizeof(int), cudaMemcpyHostToDevice));
chkerr(cudaMemcpy(d.dseq, dseq, (peelG.n) * sizeof(int), cudaMemcpyHostToDevice));
```

Figure 6: Allocating and Copying Memory on the GPU

These structures represents the input information. Next we had to allocate memory for the output information. The output of our kernel will be subgraphs generated from direct and 2-hop neighborhood of each seed vertex. One of the biggest challenges faced is that the memory cannot be assigned dynamically on GPU, therefore, we have to speculate how much information will be stored in these subgraphs and assign them accordingly. For that purpose, we created a macro called MAX_BLK_SIZE, that represents the maximum size of subgraph. We set it to 1024 currently for small to medium graphs.

Another thing is that as warp is doing work in parallel, it requires its own chunk of output information to avoid collision and dependency. For that purpose, we created really large arrays and also created offsets array d_blk_counter that will fetch only a unique part of that array based on it's warp id. Figure 7 shows the allocation of output arrays using cudaMalloc. Here all of these arrays will be used during my decomposition kernel.

```
cudaMalloc(&global_count, sizeof(unsigned int));
cudaMemset(global_count,0,sizeof(unsigned int));

cudaMalloc(&left_count, sizeof(unsigned int));
cudaMemset(left_count,0,sizeof(unsigned int));

cudaMalloc(&validblk, sizeof(unsigned int));
cudaMemset(validblk,0,sizeof(unsigned int));
// blk = the seed node + it's direct neighbors + it's two-hop
cudaMalloc(&d_blk, MAX_BLK_SIZE * WARPS * sizeof(unsigned int)
cudaMalloc(&d_blk_counter, WARPS*sizeof(unsigned int));

cudaMalloc(&d_left, MAX_BLK_SIZE*WARPS*sizeof(unsigned int));
cudaMalloc(&d_left_counter, WARPS * sizeof(unsigned int));
cudaMalloc(&d_hopSz, WARPS * sizeof(unsigned int));

cudaMalloc(&d_visited, pn * WARPS * sizeof(int)); // 40 millic
cudaMalloc(&d_count, pn * WARPS * sizeof(int));

cudaMemset(d_blk_counter, 0, WARPS*sizeof(unsigned int));
cudaMemset(d_left_counter, 0, WARPS*sizeof(unsigned int));
cudaMemset(d_hopSz, 0, WARPS * sizeof(unsigned int));
cudaMemset(d_visited, 0, pn * WARPS * sizeof(unsigned int)); /
cudaMemset(d_count, 0, pn * WARPS * sizeof(unsigned int));
```

Figure 7: Allocation and setting of memory for output information

Furthermore, as we discussed earlier, each seed vertex is assigned to one warp in our implementation. However, in case of medium to large graph, the number of nodes will much greater than the warps present. In that case, we used a for loop to divide them into batches and call the kernel iteratively to process all of them. Figure (8) shows this iteration and the kernel launch. BLK_NUMS is a macro which represents the number of blocks and BLK_DIM represents the number of threads in each block. The macro WARPS_PER_BLK can then be defined by dividing BLK_DIM by 32.

```
    for (int i = 0; i < (pn/WARPS)+1; i++)
    {
        decompose<<<BLK_NUMS, BLK_DIM>>>(i, plex_pointers, graph_pointers, degen_pointers
    }
```

Figure 8: Kernel Launch

Inside the decompose kernel, we first computed the global thread id, warp id, and lane id (thread id within the current warp). We also calculated the memory pointer assigned to the current warp using counter variables. Figure 9 illustrates this process.

```
unsigned int global_index = blockIdx.x * blockDim.x + threadIdx.x;
unsigned int warp_id = (global_index / 32);
unsigned int lane_id = threadIdx.x % 32;

//__shared__ unsigned int sh_buf[32 * 112];

if ((warp_id+WARPS*i) >= g.n) return;
//printf("Currently in thread %d of warp %d\n", lane_id, warp_id);

//unsigned int *neibors = sh_buf + (threadIdx.x / 32) * 112;
//int degreeCount = 0;
int vstart = d.dseq[warp_id+WARPS*i];
int idx;
unsigned int* blkBase = d_blk + warp_id * MAX_BLK_SIZE; // d_blk = [2048*1280]
unsigned int* counterBase = d_blk_counter + warp_id;

unsigned int* leftBase = d_left + warp_id * MAX_BLK_SIZE;
unsigned int* left_counter = d_left_counter + warp_id;
unsigned int* hopSz = d_hopSz + warp_id;

unsigned int *visitedBase = visited + warp_id * g.n;
unsigned int *countBase = count + warp_id * g.n;
```

Figure 9: Inside the Kernel: Calculating memory pointers

Next, we calculated the direct neighbors using a for loop and stored it in blk (current subgraph) or left (nodes already processed) based on the node's position. Figure 10 represents the process.

```
if (lane_id == 0)
{
  idx = atomicAdd(&counterBase[0], 1);
  blkBase[idx] = vstart; // seed vertex
  visitedBase[vstart] = 1;
  //printf("Block Size is %d in warp %d\n", counterBase[0], warp_id);
}
for (int i = lane_id; i < g.degree[vstart]; i += 32)
{
  int nei = g.neighbors[g.offsets[vstart] + i];

  //if (!visitedBase[nei]) // position of one-hop
  //{
    if (d.dpos[vstart] < d.dpos[nei])
    {
      idx = atomicAdd(&counterBase[0], 1);
      blkBase[idx] = nei;
    }
    else
    {
      idx = atomicAdd(&left_counter[0], 1);
      leftBase[idx] = nei;
    }
    visitedBase[nei] = 1;
  //}
}
```

Figure 10: Calculating the direct neighbors of seed vertex

Next, we calculate two-hop neighbors using two nested for-loops. In the first for loop, we calculate how many times each node is present in the two-hop neighborhood of the current seed vertex and storing it in countBase. Then, if that count is greater than a certain threshold as discussed above, only then it's included in the current subgraph or left. Figure 11 illustrates this process in code. __syncwarp() synchronizes all the threads within a warp and only moves when all the threads in a warp have reached that point.

```
for (int i = lane_id; i < g.degree[vstart]; i+=32
{
  int nei = g.neighbors[g.offsets[vstart] + i];
  if (d.dpos[vstart] < d.dpos[nei]){
    for (int j = 0; j < g.degree[nei]; j++)
    {
        int twoHop = g.neighbors[g.offsets[nei]+j
        if (!visitedBase[twoHop]) // blk -> direc
        {
          atomicAdd(&countBase[twoHop], 1);
        }
    }
}
}

__syncwarp(0xFFFFFFFF); //count = [3 4 5]
```

(a)

```
for (int i = lane_id; i < g.degree[vstart]; i+=32)
{
  int nei = g.neighbors[g.offsets[vstart] + i];
  if (d.dpos[vstart] < d.dpos[nei])
  {
    for (int j = 0; j < g.degree[nei]; j++)
    {
      int twoHop = g.neighbors[g.offsets[nei]+j];
      int old = atomicCAS(&visitedBase[twoHop], 0, 1);
      if (old == 0)
      {
        if (d.dpos[vstart] < d.dpos[twoHop])
        {
          if (countBase[twoHop] >= lb_2k + 2)
          {
            idx = atomicAdd(&counterBase[0], 1);
            blkBase[idx] = twoHop;
          }
        }
        else
        {
          if (countBase[twoHop] >= lb_2k + 3)
          {
            idx = atomicAdd(&left_counter[0], 1);
            leftBase[idx] = twoHop;
          }
        }

    }
    }
}
}

__syncwarp(0xFFFFFFFF);
```

(b)

Figure 11. Calculating Two-Hop Neighborhood of the seed vertex

Finally, if our current subgraph has more than q nodes, only then it will be considered a valid subgraph and stored for further computation. Also, in the end, we cleaned the visited, count, counterBase, and leftCounter to reuse them for the next iteration. Figure 12 illustrates the process in code. The global variables are atomically added to avoid race conditions and collision.

```
if (lane_id == 0)
{
  if (counterBase[0] >= p.lb){
    //printf("Number of direct neighbors
    atomicAdd(&validblk[0], 1);
    atomicAdd(&global_count[0], counterBa
    atomicAdd(&left_count[0], left_counte
  }
}
CLEAN:
for (int i = lane_id; i < g.n; i += 32)
{
  visitedBase[i] = 0;
  countBase[i] = 0;
}

if (lane_id == 0)
{
  //printf("Total number of nodes in blk
  counterBase[0] = 0;
  left_counter[0] = 0;
}

__syncwarp(0xFFFFFFFF);
```

Figure 12: Incrementing the valid blocks and cleaning the variables

3. Test Plan

To evaluate our work, we focused on two key goals: correctness and performance. We wanted to ensure that our GPU-accelerated version of the algorithm (using CUDA) produced the same results as the original CPU implementation, and also that it did so faster, especially on large, real-world graphs.

3.1. Parameters and Setup

We used Cheaha server to test our code and utilize their pascal GPU nodes. We used nvidia-smi command to check the characteristics of our GPU. Figure 13 shows the Tesla P100 PCIE 16GB GPU assigned for our computation. This GPU has 16 GB global memory and 56 Streaming Multiprocessors. We launched our kernel with 56 blocks and 1024 threads in each block, thus providing us with a total of 1792 warps. Thus, 1792 works units were doing computations on original graph in parallel.

```
[kefan@c0113 Maximal-kPlex-cuda]$ nvidia-smi
Mon May 12 17:15:44 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.90.07              Driver Version: 550.90.07      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla P100-PCIE-16GB           On  |   00000000:85:00.0 Off |                    0 |
| N/A   28C    P0             25W /  250W |       0MiB /  16384MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

Figure 13: Nvidia Tesla P100 used on cheaha server

For testing, we used 4 real-world datasets (small to medium) with different values of k and q. Figure 14 summarizes the test cases we used for our algorithm.

Dataset	n	m	k	q
jazz	198	2742	4	12
wiki-vote	7116	100763	3	20
soc-epinions	75879	405740	4	30
email-euall	265009	364481	3	12

Figure 14: Datasets used with k and q value

3.2. Datasets

We tested our implementation on a variety of real-world graph datasets in .bin format, ranging from small and dense to extremely large and sparse:

- **jazz.bin**: A small, dense social network of jazz musicians (198 nodes). This dataset helped us verify correctness and measure any fixed overhead in the GPU version. Due to the high connectivity, we expected many k-plexes.

- **wiki-vote.bin**: A medium-sized network from Wikipedia voting data (7,116 nodes). This graph let us test the GPU's handling of moderately sized inputs—large enough to matter, but still manageable.
- **soc-epinions.bin**: A large network of trust relationships (75,879 nodes). It's relatively sparse, which allowed us to test how effective pruning is at this scale, and how well our implementation scales with graph size.
- **email-euall.bin**: A very large email communication graph (265,009 nodes). This one pushed the limits of our approach. It tested both performance and memory management on large, sparse graphs with hundreds of thousands of nodes.

For each of these datasets, we calculated the total number of valid blocks, total number of nodes present in all the subgraphs, and total number of nodes present in all the left arrays. The last two parameters were used for verification purposes to check whether the correct number of nodes are stored in our subgraphs.

4. Results and Analysis:

The GPU-accelerated version ran much faster than the original CPU algorithm in every test. Table 1 gives a breakdown of the results for each dataset, showing the graph size, the size after pruning, and how long each version took to run.

Dataset	**Total Nodes**	**Valid Nodes (after prune)**	**CPU Time (ms)**	**GPU Time (ms)**	**Speedup**
jazz.bin	198	158	99.7	0.5	199×
soc-epinions.bin	75,879	3,857	152.7	55.9	3×
wiki-vote.bin	7,116	2,153	91.7	26.0	4×
email-euall.bin	265,009	2,776	137.0	12.1	11×

Table 1: CPU vs GPU Implementation Results

Figure 15 illustrates the visual representation of our CPU vs GPU implementation results.

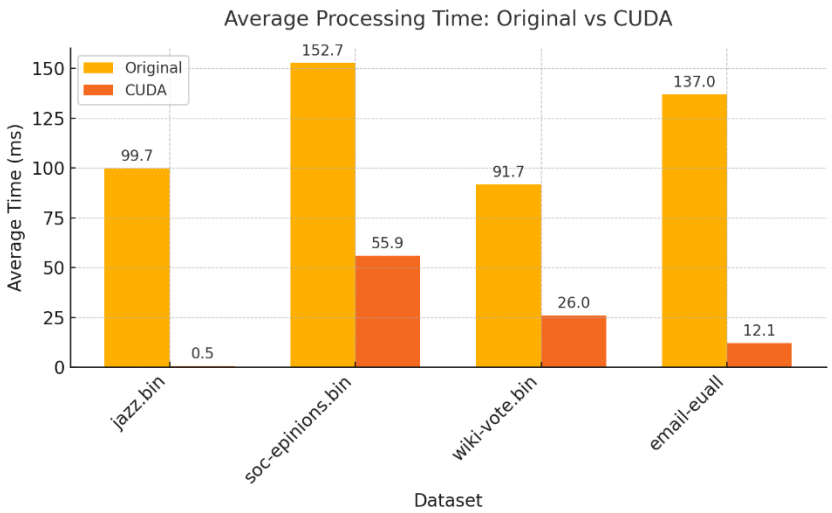

Figure 15: CPU vs GPU Implementation Results

The results show that our CUDA-based approach can dramatically cut down runtime. Take the Jazz network, for example, it's a small but dense graph, and the GPU finds all maximal 4-plexes in just 0.5 milliseconds on average, compared to about 99.7 milliseconds on the CPU. That's nearly a 200× speedup. This huge gain comes partly from the small size of the problem, which keeps GPU overhead low, and partly because the graph's structure makes it easy to split the work across many parallel threads. It's a clear sign that our GPU implementation handles even small-scale tasks efficiently, with very little launch overhead and solid parallel performance.

On larger graphs, the GPU still delivered strong speedups, though the gains varied: about 3× on soc-epinions, 4× on wiki-vote, and up to 11× on email-EU. Several factors help explain this variation.

First, pruning had different levels of effectiveness depending on the dataset. For example, the Epinions graph started with over 75,000 nodes, but after pruning, only about 3,800 were left to search so the GPU and CPU were both working on a much smaller graph. Since the CPU can handle a graph of that size reasonably well, the GPU's advantage was smaller, around 3×. On the other hand, the email-EU graph, though pruned to about 2,700 nodes, likely had a more complex structure that required deeper and more frequent branching. In that case, the GPU's ability to explore multiple branches in parallel really paid off, giving an 11× speedup.

Another factor is how well the work is distributed across the GPU. Our implementation uses a Bron–Kerbosch-style search, which doesn't always split the workload evenly. Some pivot nodes result in a lot of computation and many recursive calls, while others finish quickly. In our design, each GPU warp handles one pivot. If some warps finish early and

others are stuck on heavier tasks, parts of the GPU sit idle. This kind of imbalance likely affected the Epinions case, where a few high-degree nodes dominated the search. In contrast, the wiki-vote and email-EU datasets saw better workload distribution, allowing more of the GPU to stay active, which contributed to their higher speedups.

Finally, the GPU's coordination overhead and memory access patterns also matter. Even after pruning, large graphs still require each warp to manage arrays of size n for things like visited flags and counters. That means lots of reads and writes to global memory often with atomic operations which can slow things down if the accesses aren't well-aligned. In smaller or denser graphs where the actual computation dominates, this overhead is less noticeable. But in some cases, it does limit the full potential speedup. Even so, the GPU consistently outperformed the CPU across all our tests.

As for correctness, the GPU algorithm produced the same sets of maximal k-plexes as the CPU version, we verified the counts on smaller graphs to be sure. While we don't list results for the larger datasets (since the outputs are massive), the consistent behavior across all tests gives us confidence in the accuracy of the parallel implementation. Figure 16 shows the results from both GPU and CPU implementation.

```
[kefan@c0113 Maximal-kPlex-cuda]$ ./main ./dataset/jazz.bin -k 4 -q 12
file: ./dataset/jazz.bin
k=4,q=12
n=198, m=5484 (undirected)

pn = 172
Start copying graph to GPU....
Done copying graph to GPU....
Total Time Elapsed: 0.584768 ms
Total Valid Blocks: 158, Total Block Size: 5875, Total Left Size: 3142

Kernel Launch Successfully
```

(a)

```
[kefan@c0113 Parallel]$ ./PlexEnum ../Dataset/jazz.bin -k 4 -q 12 -t 8
file: ../Dataset/jazz.bin
k = 4,q = 12
Thread number : 8
Timeout threshold: 0.100
n=198, m=5484 (undirected)
Time of read graph:0.014
peelG.n = 172
Total valid blocks 158, total block size: 5875, total left size: 3142
Running time 0.284s
```

(b)

Figure 16: Results Verifications Between GPU(a) and CPU(b)

Overall, the project shows that using a GPU can significantly speed up the enumeration of maximal k-plexes. Even with a fairly straightforward adaptation of the decomposition process in branch-and-bound search to run in parallel across GPU warps, we saw speedups ranging from 3× to nearly 200×, depending on the graph. That kind of boost makes it practical to analyze much larger networks in reasonable time. For example, our largest test graph with 265,000 nodes ran in just 12 milliseconds on the GPU, compared to 137 milliseconds on the CPU. And for graphs with millions of edges, the performance gains become even more important.

5. Future Work

There are several promising directions to build on this work and make it even faster and more flexible:

5.1. Full GPU Parallelism:

Right now, we're only parallelizing the decomposition part of the Maximal k-Plex Enumeration Algorithm. In the future, we intend to parallelize the entire recursive search on GPU which can lead to massive improvements in enlisting maximal k-Plexes in real-world graphs. We will also try to implement adaptability into our algorithm, so as to choose other implementations based on graph characteristics.

5.2. Better Load Balancing:

Currently, many of our warps go idle if their assigned seed vertex cannot make a subgraph that's valid and is pruned. They are inactive until all other warps have done their work. This causes huge inefficiency as on some graphs, more than two-third of the GPU's resources aren't being utilized. We'll try to implement new strategies to improve this and avoid warp idleness as much as possible.

5.3. Memory and Data Structures Improvements:

Our current setup uses large boolean and integer arrays per warp, which can consume a lot of memory and cause inefficient access patterns. Replacing these with more compact structures like bitsets could help. Bit-parallel operations (using 32- or 64-bit words to represent sets) would speed up things like intersection checks and missing-edge counts. We could also use shared memory more effectively for frequently accessed data, and reduce reliance on atomic operations to cut down on bottlenecks.

6. Conclusion

This project showed that GPU acceleration can make a big difference in finding maximal k-plexes, especially in large graphs. By combining recent improvements in the underlying search algorithm with the power of a Tesla P100 GPU, we achieved major speedups over the CPU version. With further work like deeper parallelism, better load balancing, and memory optimizations this approach can become an even more powerful tool for real-world network analysis.

7. References

1. Q. Dai, R. H. Li, H. Qin, et al. "Efficient Enumeration of Large Maximal k-Plexes." Proc. of the 28th International Conference on Extending Database Technology (EDBT), 2025. (Introduces the branch-and-bound algorithm for maximal k-plexes, with pruning techniques and an OpenMP parallel version)
2. Github of the original algorithm implementation (https://github.com/chengqihao/Maximal-kPlex?tab=readme-ov-file#tested-environment)
3. https://www.uab.edu/it/home/research-computing/cheaha
4. OpenAI ChatGPT used to assist in analysis and report writing (https://chatgpt.com/)